



Programação II

2025/2026 - João Rebelo - ISTEC - WPF



Programação II

WPF (Windows Presentation Foundation)

WPF

- **FRAMEWORK** - Construção de Aplicações WINDOWS
- Permite separar a Interface (UI em XAML) da Lógica do programa (C#)
- Data Binding, ou *como não estar preocupado com informações em caixas*; atualização automática de informação.
- Organização Flexível de componentes de interação com utilizador
 - Grid (organização em tabela)
 - StackPanel (empilhar objetos verticalmente ou horizontalmente)
 - Etc...
- Suporte avançado a Estilos e Animações (criação de conteúdos interativos)
- Lógica de programação O.O. e orientada a Eventos

WPF - Data Binding - Automação

- Princípio que requer a utilização de PROPRIEDADES
- PROPRIEDADE:
 - Alternativa MS aos Getters e Setters
 - Permite gerir atributos Privados de forma controlada, tal como Getters e Setters
 - Bloco onde podemos definir:
 - A parte do GETTER → GET
 - A parte do SETTER → SET

```
public class DemoProp {  
    private string _fraseNaoVazia;
```

```
    public void setFraseNaoVazia(string frase) {  
        _fraseNaoVazia = frase;  
        if (_fraseNaoVazia.Length == 0) {  
            _fraseNaoVazia = "A frase não pode ser vazia";  
        }  
    }  
  
    public string getFraseNaoVazia() {  
        return _fraseNaoVazia;  
    }  
}
```

Controlo por Setter/Getter

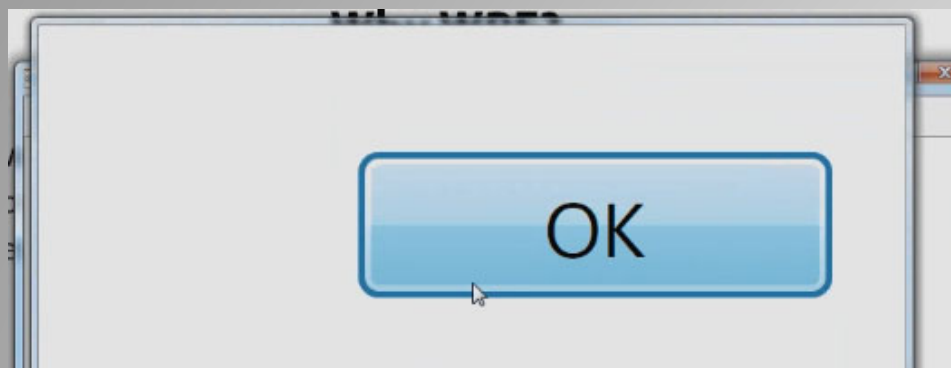
```
public string FraseNaoVazia {  
    get { return _fraseNaoVazia; }  
    set {  
        _fraseNaoVazia = value;  
        if (_fraseNaoVazia.Length == 0) {  
            _fraseNaoVazia = "A frase não pode ser vazia";  
        }  
    }  
}
```

Porquê WPF ?

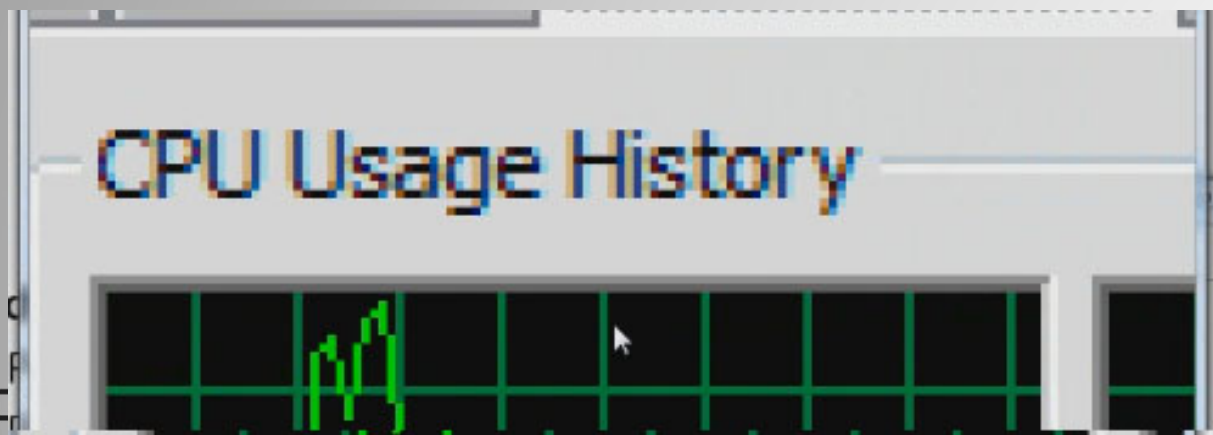
- **Ao desenvolver em WPF** não nos preocupamos com a resolução dos monitores/displays: escalabilidade das aplicações, relativamente à qualidade cada vez maior dos dispositivos de visualização.
- **Independente das plataformas** - 1 jogo WPF no pc tem a mesma qualidade que 1 jogo WPF na XBOX ou dispositivo tablet, Android, etc.
- **Lições Aprendidas:**
 - Aplicações visualmente apelativas e coerentes. Ex: um sítio de uma empresa deve refletir a própria empresa (Branding)
 - Linguagens Universalmente Aceites: Linguagens de Marcação (HTML, XML, etc.)
- **Integração: Ao desenvolver em WPF**, juntamos a versatilidade de branding (característico do HTML/CSS) com a eficiência de gestão de dados (aplicações tradicionais) e a velocidade de gestão de gráficos (caraterística de jogos em DirectX)

Exemplo Pixelização vs Alta Resolução

- Utilizar o “Magnifier” do Windows para ver uma aplicação GDI32 e uma aplicação WPF



- o “Magnifier” numa aplicação WPF



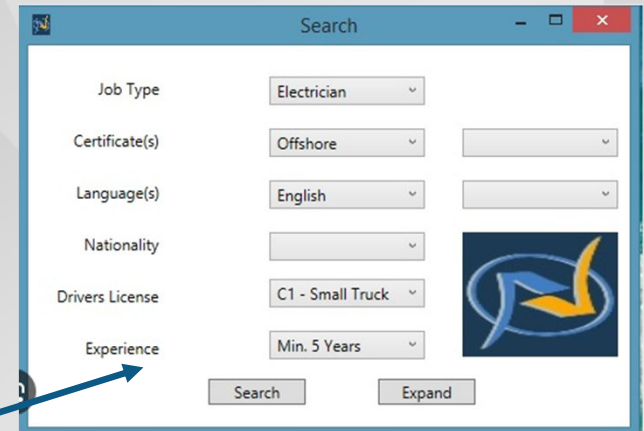
- o “Magnifier” numa aplicação tradicional Windows FORMS

Como? Separar a Lógica da UI

- **Organização da UI** com recurso a uma linguagem de marcação própria, XAML - eXtensible Application Markup Language, que permite:
 - Organizar Interfaces Gráficas
 - Conjunto de objetos pré-definidos para interação (label, text, button...)
 - Layouts
 - Estilos
 - Animações
 - Definição de Comportamentos (Invocar/Chamar Eventos)
 - Bindings
- **Organização da Lógica** - com recurso a abordagens O.O. (C# ou VB) e com a possibilidade de gerir Eventos

Princípio: Páginas “Inteligentes”

```
<Window x:Class="WpfApp3.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/2010/dll"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:WpfApp3"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="640"
>
    <Grid>
        <Slider x:Name="sldVolume" Minimum="0" Maximum="100" Value="50" />
        <Label x:Name="lblDisplay" Content="Volume: 50" />
        <Button x:Name="btnMute" Content="Mute" />
    </Grid>
</Window>
```



Página XAML (2 vistas)

mensagens

informação

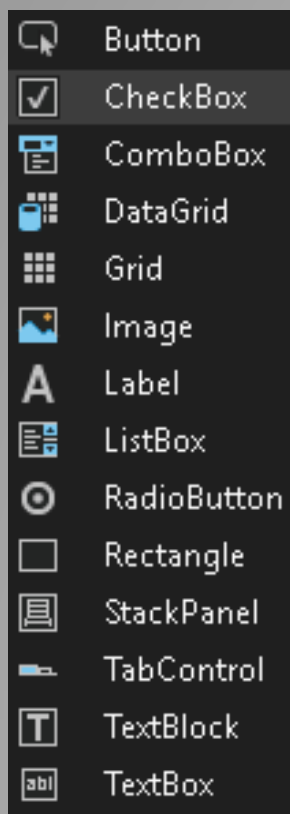
Página C#/VB

```
1 reference
private void btnMute_Click(object sender, RoutedEventArgs e) {
    sldVolume.Value = 0;
}

1 reference
private void sldVolume_ValueChanged(object sender, RoutedPropertyChangedEventArgs<double> e) {
    lblDisplay.Content = $"Antes: {e.OldValue} / Após: {e.NewValue}";
}
```


WPF - Interação com Utilizador

Disponível um conjunto de objetos UI pré-definidos



Cada Objeto:

- Tem Propriedades
 - Sabe fazer coisas
 - Pode responder a mensagens (definir Eventos)
- Conceito O.O. tradicional

(Alguns dos objetos)

WPF - Conceito Code-Behind

- **Cada Página WPF contém:**
 - Uma descrição dos componentes e respetivo local: XAML
 - Como respondem os objetos a mensagens que podem ouvir: C# em página Code-Behind

```
<Grid>
    <Label Content="Nome:" HorizontalAlignment="Left" Margin="0,0,0,0" />
    <TextBox x:Name="txtNome" HorizontalAlignment="Left" Width="150" />
    <Button x:Name="btnMostrar" Click="BtnMostrar_Click" />
</Grid>
</Window>
```



XAML (especialização do XML, tal como o HTML)

Code-Behind
(C# ou VB)

```
private void BtnMostrar_Click(object sender, RoutedEventArgs e) {
    MessageBox.Show("Digitaste:" + txtNome.Text);
}
```

WPF - Programação de Eventos

- **Evento**

- Resposta de um Objeto...
- ...sempre que recebe uma determinada Mensagem

- Distinção Fundamental entre Função e Evento:

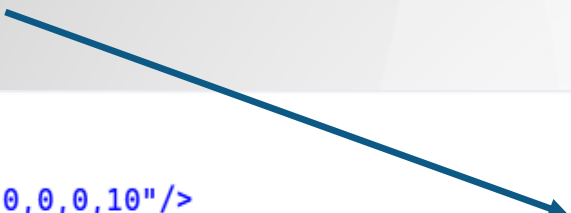
- Na função, enviamos nós os parâmetros
- No evento, é-nos devolvido o contexto de um objeto como parâmetro desse evento !?=#\$!#\$ ☹
- **W.T.H.I.A.Gw? (Marty Macfly in “Regresso ao Futuro”)**

WPF - Programação de Eventos

- **Exemplo de Eventos vs. Função/Procedimento (Parte 1)**
- **Função** - dizer a um objeto para fazer algo. Ex: pedir a uma string para fazer um Trim()
resultado = Nome.Trim();
- **Evento** - enviar uma mensagem a um objeto. Ele deve responder de acordo com o que lhe foi ensinado.

No XAML criar a assinatura de evento...

```
<Grid Margin="20">
  <StackPanel>
    <TextBlock Text="Volume" FontSize="18" Margin="0,0,0,10"/>
    <Slider x:Name="sliderVolume" Minimum="0" Maximum="100" ValueChanged="sliderVolume_ValueChanged"/>
    <TextBlock x:Name="txtValor" Text="Valor: 0" FontSize="20" Margin="0,20,0,0"/>
  </StackPanel>
</Grid>
```



WPF - Programação de Eventos

- Exemplo de Eventos vs. Função/Procedimento (Parte 2)

```
//A função de evento é invocada sempre que dissermos ao slider  
//que lhe vamos alterar a posição,  
//ou seja, sempre que o valor do slider for alterado.  
//O evento é ValueChanged (mensagem que vamos mandar)  
//e a função de evento é sliderVolume_ValueChanged (o que objeto  
//vai fazer quando ouvir esta mensagem)
```

1 reference

```
private void sliderVolume_ValueChanged(  
    object sender,  
    RoutedPropertyChangedEventArgs<double> e) {  
    // Vamos alterar o texto do txtValor para mostrar o valor do slider  
    // sempre que o utilizador disser ao slider: estou a mexer no teu cursor  
    txtValor.Text = $"Valor: {sliderVolume.Value}";  
}
```

WPF - Objetos

- O Framework WPF permite diferentes tipos de objetos
 - Embutidos (built-in)
 - Disponíveis via “*library*”
- Ambos:
 - “*Object Oriented*” com Herança implícita-.
 - Utilizáveis de forma hierárquica ou independente
 - Suportam Propriedades (atributos públicos)
 - Suportam Comandos (métodos)
 - Respondem a Mensagens (disparam eventos)

WPF - Objetos - Continuação

- Exemplos de Objetos Embutidos:
 - Botão
 - TextBox, Label
 - Etc.
- Exemplos de Objetos em “*Libraries*”
 - DispatcherTimer
 - Random
 - Etc.

WPF - Dois Exemplos de Aula

- Uma Calculadora IMC

MainWindow

Peso (kg): 90

Altura (m): 1,80

Calcular IMC

Resultado: 27,78

- Um configurador de cores RGB

MainWindow

Red

Green

Blue

RGB(58, 98, 128)

Dúvidas e Sugestões

- jrebelo@my.istec.pt

(João Rebelo)